

Reviewer Pack — Minimal Geometric Entropy of Tensors and Circuit Size Inequa...

Entry cb5fbc0beba2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 07:59:45 UTC

Minimal Geometric Entropy of Tensors and Circuit Size Inequality

Entry ID: cb5fbc0beba2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 07:59:45 UTC

1. Conjecture

Field A (mathematical branch): Geometric Information Theory **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For every Boolean function f , the minimal geometric entropy of its tensor representation is logarithmically correlated with the size of the smallest circuit computing f , i.e., $H_{\min}(g_f) = \Theta(\log(s(f)))$ where $s(f)$ is the size of the smallest circuit for f .

Rationale (proposer's reasoning):

Geometric information theory provides a framework for understanding complexity in terms of geometric objects. Tensors can represent Boolean functions, and their geometric entropy measures the amount of information needed to describe the function. This conjecture suggests that this geometric information directly relates to the classical complexity measure of circuit size.

Taxonomy category: GeometricEntropy (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a0ad10cf57830686

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every Boolean function, if the correlation coefficient between minimal geometric entropy and logarithm of circuit size is greater than or equal to 0.7 for all 30 seeds, then support the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import product, combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def tensor_representation(f, n):
        tensor = [[f[i] if j == i else 0 for j in range(2**n)] for i in range(2**n)]
        return tensor

    def geometric_entropy(tensor):
        total = sum(tensor[i][j] for i in range(len(tensor)) for j in range(len(tensor[0])))
        entropy = 0
        for i in range(len(tensor)):
            row_sum = sum(tensor[i])
            if row_sum > 0:
                p = Fraction(row_sum, total)
                entropy -= p * math.log2(p)
        return entropy

    def circuit_size(f, n):
        # Simplified DPLL algorithm to find the size of the smallest circuit
        def dpll(cnf, assignment):
            if not cnf:
                return True
            unit_clause = next((c for c in cnf if len(c) == 1), None)
            if unit_clause:
                literal = unit_clause[0]
                new_assignment = assignment.copy()
                new_assignment[literal] = literal > 0
                if dpll([c for c in cnf if literal not in c], new_assignment):
                    return True
```

```

        new_assignment[literal] = not literal > 0
        if dpll([c for c in cnf if -literal not in c], new_assignment):
            return True
        return False
    pure_literal = next((l for l in range(1, n+1) if (l not in assignment and -l not in assignment)), None)
    if pure_literal:
        new_assignment[pure_literal] = True
        if dpll(cnf, new_assignment):
            return True
        new_assignment[pure_literal] = False
        if dpll(cnf, new_assignment):
            return True
        return False
    literal = next((l for l in range(1, n+1) if l not in assignment), None)
    new_assignment[literal] = True
    if dpll([c for c in cnf if literal not in c], new_assignment):
        return True
    new_assignment[literal] = False
    if dpll(cnf, new_assignment):
        return True
    return False

def cnf_to_clauses(cnf):
    clauses = []
    for clause in cnf:
        if isinstance(clause, list):
            clauses.append(clause)
        else:
            clauses.append([clause])
    return clauses

cnf = generate_cnf(f, n)
if cnf is None:
    return 0
assignment = {}
size = 0
while not dpll(cnf, assignment):
    literal = next((l for l in range(1, n+1) if l not in assignment), None)
    new_assignment = assignment.copy()
    new_assignment[literal] = True
    if dpll(cnf, new_assignment):
        assignment[literal] = True
        size += 1
    else:
        new_assignment[literal] = False
        if dpll(cnf, new_assignment):
            assignment[literal] = False
            size += 1
return size

def generate_cnf(f, n):
    # Simplified CNF generation for a random Boolean function
    cnf = []
    for i in range(2**n):
        clause = []
        for j in range(n):

```

```

        if f[i] == 0:
            clause.append(-j-1)
        else:
            clause.append(j+1)
        cnf.append(clause)
    return cnf

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    f = generate_random_boolean_function(n)
    tensor = tensor_representation(f, n)
    entropy = geometric_entropy(tensor)
    s_f = circuit_size(f, n)
    if s_f == 0:
        continue
    results.append({"n": n, "entropy": entropy, "s_f": s_f})

if not results:
    return {
        "metric_name": "geometric_entropy",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

mean_entropy = sum(result["entropy"] for result in results) / len(results)
mean_s_f = sum(result["s_f"] for result in results) / len(results)
correlation_coefficient = 0
if mean_s_f != 0:
    correlation_coefficient = sum((result["entropy"] - mean_entropy) * (math.log2(result["s_f"] - 1) - math.log2(mean_s_f))) / len(results)

return {
    "metric_name": "geometric_entropy",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(result["n"] for result in results),
    "conjecture_holds": correlation_coefficient >= 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

```

```

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
elif any(not result["conjecture_holds"] for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e3803bc9.py", line 154, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e3803bc9.py", line 119, in run_trial
    s_f = circuit_size(f, n)
           ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e3803bc9.py", line 85, in circuit_size
    while not dpll(cnf, assignment):
               ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e3803bc9.py", line 55, in dpll
    new_assignment[pure_literal] = True
    ^^^^^^^^^^^^^
UnboundLocalError: cannot access local variable 'new_assignment' where it is not associated with a value

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to calculate the correlation coefficients. | next: Review and fix the error in the test code that caused the crash, then rerun the test to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13783
2	preregistration	ollama_remote	glm4:latest	0	0	9722
3	novelty	ollama_remote	glm4:latest	0	0	8213

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	8468
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19135
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15010
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17516
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17878
9	judge	ollama_remote	glm4:latest	0	0	20289

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 130013 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/cb5fbc0beba2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cb5fbc0beba2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/cb5fbc0beba2.tar.gz` (if generated)